Kubernetes Lab Workbook

Step-by-step labs aligned to ShadowXLab Modules 1–3

Beginner → Engineer · 12 hands-on labs · Linux + Windows

Format: PDF · ~24 pages
Audience: Beginner → Engineer → Enterprise
Author: ShadowXLab Curriculum Team

© ShadowXLab · Enterprise Kubernetes Academy

Lab 1 — Run your first container

Module 1 · 10 min · Beginner

Objective. Pull and run an nginx container, expose port 80, inspect the running process.

Commands:

```
docker pull nginx:alpine
docker run -d --name web -p 8080:80 nginx:alpine
curl http://localhost:8080
docker exec web nginx -v
```

Expected output. HTTP 200 with the nginx welcome page. nginx -v reports the version.

Lab 2 — Build a Dockerfile

Module 2 · 20 min · Beginner

Objective. Build a Python app image with a multi-stage Dockerfile.

Commands:

```
# Dockerfile
FROM python:3.12-slim AS base
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]

# Build
docker build -t shadowx/app:1.0 .
docker images | grep shadowx
```

Expected output. Image built and tagged. docker images shows ~80 MB image.

Lab 3 — Push to a registry

Module 2 · 15 min · Engineer

Objective. Tag and push the image to Docker Hub or GHCR.

Commands:

```
docker tag shadowx/app:1.0 ghcr.io/<you>/app:1.0
echo $GHCR_TOKEN | docker login ghcr.io -u <you> --password-stdin
docker push ghcr.io/<you>/app:1.0
```

Expected output. Push succeeds; image visible in registry UI.

© ShadowXLab · Enterprise Kubernetes Academy

Lab 4 — kubectl & first Pod

Module 3 · 10 min · Beginner

Objective. Spin up a one-shot pod and view its logs.

Commands:

```
kubectl run hello --image=busybox --restart=Never -- echo "hello shadowx"
kubectl logs hello
kubectl delete pod hello
```

Expected output. STDOUT prints hello shadowx then pod completes.

Lab 5 — Deployment + Service

Module 3 · 25 min · Engineer

Objective. Deploy nginx with 3 replicas, expose as ClusterIP, hit it from a debug pod.

Commands:

```
kubectl create deploy web --image=nginx:alpine --replicas=3
kubectl expose deploy web --port=80
kubectl run curl --rm -it --image=curlimages/curl -- sh
# inside debug pod:
curl http://web
```

Expected output. Three pods Running. curl http://web returns nginx welcome page.

Lab 6 — Diagnose a CrashLoopBackOff

Module 3 · 15 min · Engineer

Objective. Deploy a pod that fails on start and use logs/events to find the cause.

Commands:

```
kubectl run bad --image=busybox --restart=Never -- sh -c "exit 1"
kubectl get pod bad -w
kubectl describe pod bad
kubectl logs bad --previous
```

Expected output. Pod enters CrashLoopBackOff. Events show 'Back-off restarting failed container'.

© ShadowXLab · Enterprise Kubernetes Academy

Lab 7 — Windows worker node check

Module 3 · 20 min · Enterprise

Objective. Schedule a Windows pod onto a Windows node (requires hybrid cluster).

Commands:

```
kubectl get nodes -L kubernetes.io/os
kubectl apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata: {name: win-iis}
spec:
nodeSelector: {kubernetes.io/os: windows}
tolerations: [{key: os, value: windows, effect: NoSchedule}]
containers:
- name: iis
image: mcr.microsoft.com/windows/servercore/iis
EOF
```

Expected output. Pod schedules on a Windows node and serves IIS default page.

Lab 8 — RBAC for a junior SRE

Module 3 · 15 min · Enterprise

Objective. Grant read-only access to a single namespace.

Commands:

```
kubectl create ns billing
kubectl create role viewer --verb=get,list,watch --resource=pods,services -n billing
kubectl create rolebinding alice-viewer --role=viewer --user=alice -n billing
kubectl auth can-i list pods -n billing --as=alice
```

Expected output. can-i returns yes for billing, no for default.

© ShadowXLab · Enterprise Kubernetes Academy